

Paradigmas de Programación

TRABAJO PRÁCTICO N° 2

1. Cálculo Lambda

1.1. Teoría Lambda

Ejercicio 1.1. ¿Qué son las reglas de reducción? ¿Por qué se utilizan?

Ejercicio 1.2. ¿Qué significa la forma normal de una función lambda?

Ejercicio 1.3. ¿Toda función se puede expresar como un cálculo lambda? Justifique su respuesta.

Ejercicio 1.4. ¿Puede existir una λ -expresión bien formada que nunca llegue a forma normal?

Ejercicio 1.5. Explique qué ideas del λ -cálculo aparecen en lenguajes reales (por ejemplo: funciones anónimas, clausuras, alcance léxico). Nombre 2-3 y describa el vínculo.

Ejercicio 1.6. ¿Qué es una forma normal en λ -cálculo? ¿Qué diferencia conceptual hay entre “no puedo reducir más” y “no sé cómo reducir”?

Ejercicio 1.7. ¿Qué significa que dos expresiones sean α -equivalentes? ¿Es lo mismo que “son iguales”? ¿Qué cosas no se pueden cambiar con α -renombrado?

1.2. Práctica Lambda

Ejercicio 1.8. ¿Cuáles de las siguientes expresiones son λ -expresiones correctas según la gramática vista en clase? ¿Cuáles no lo son? Justifique cada respuesta.

A. $\lambda x.x$

D. λxx

G. $\lambda x.y$

B. $\lambda x.xy$

E. $\lambda x.xy\lambda x.xy$

H. $\lambda x.\lambda y.xy$

C. $\lambda x.x$

F. $(\lambda x. ((x y) . z))$

I. $(\lambda x. (x \lambda y. y))$

Ejercicio 1.9. Para cada inciso, haga explícitos los paréntesis de las siguientes λ -expresiones. Además, indique, para cada variable, cuáles de sus ocurrencias son libres y cuáles acotadas. Indique a qué abstracción- λ está ligada cada ocurrencia no libre.

A. $(\lambda x. x y) z$

B. $\lambda x.xz) \lambda y.w \lambda w.wyzx$

C. $\lambda x.xy \lambda x.yx$

D. $(\lambda x.\lambda y.xy) (\lambda z.z w)$

E. $\lambda x.(\lambda y.x (\lambda xyx)) x$

Ejercicio 1.10. Utilizando β -reducción, simplifique las siguientes expresiones hasta una forma final (normal), si existe. Si no existe, explique por qué.

- $((\lambda x.(x\ y))(\lambda z.z))$
- $((\lambda x.((\lambda y.(x\ y))x))(\lambda z.w))$
- $((((\lambda f.((\lambda g.(\lambda x.((f\ x)\ (g\ x)))))(\lambda m.(\lambda n.(n\ m)))))(\lambda n.z))\ p)$
- $((\lambda x.(x\ x))(\lambda x.(x\ x)))$
- $((\lambda f.((\lambda g.((f\ f)g))(\lambda h.(k\ h))))(\lambda x.(\lambda y.y)))$
- $(\lambda g.((\lambda f.((\lambda x.(f\ (x\ x)))(\lambda x.(f\ (x\ x))))\ g))$

Ejercicio 1.11. Para los siguientes términos, marque la opción correcta:

- $\lambda f. (\lambda x. f (\lambda y. x\ x\ y)) (\lambda x. f (\lambda y. x\ x\ y))) (\lambda g. \lambda y. y) (\lambda h.h)$
 - $(\lambda f. (\lambda x. f (\lambda y. x\ x\ y)) (\lambda x. f (\lambda y. x\ x\ y))) (\lambda g. \lambda y. y)$
 - $\lambda h.h$
 - $(\lambda x. (\lambda g.\lambda y.y) (\lambda y. x\ x\ y)) (\lambda x. (\lambda g. \lambda y. y) (\lambda y. x\ x\ y))$
 - $\lambda x. (\lambda g.\lambda y.y) (\lambda y. x\ x\ y)$
 - NINGUNO
- $(\lambda f. (\lambda x. f (\lambda y. x\ x\ y)) (\lambda x. f (\lambda y. x\ x\ y))) (\lambda g. g) (\lambda h.h)$
 - $(\lambda f. (\lambda x. f (\lambda y. x\ x\ y)) (\lambda x. f (\lambda y. x\ x\ y))) (\lambda g. g)$
 - $\lambda h.h$
 - $(\lambda x. (\lambda g.g) (\lambda y. x\ x\ y)) (\lambda x. (\lambda g.g) (\lambda y. x\ x\ y))$
 - $\lambda x. (\lambda g.g) (\lambda y. x\ x\ y)$
 - NINGUNO

2. Programación Funcional

2.1. Teoría

Ejercicio 2.1. ¿Qué es la programación funcional y qué características presenta?

Ejercicio 2.2. ¿Qué características presenta un programa escrito en un lenguaje funcional con respecto a un programa escrito en un lenguaje imperativo? ¿Qué rol cumple la computadora?

Ejercicio 2.3. ¿Cuál es el concepto de variables en los lenguajes funcionales?

Ejercicio 2.4. Enumere y explique brevemente cuáles son las diferencias entre un lenguaje funcional, un lenguaje lógico y un lenguaje procedimental.

Ejercicio 2.5. ¿En qué situaciones cree que sería mejor utilizar el paradigma funcional?

Ejercicio 2.6. ¿El paradigma funcional se basa en el funcionamiento de la máquina de Von Neumann?

Ejercicio 2.7. Explique qué es el Cálculo Lambda y por qué los lenguajes funcionales se basan en él.

Ejercicio 2.8. Explique qué es la transparencia referencial.

Ejercicio 2.9. Haskell es un lenguaje de programación funcional con un sistema de tipos (solo existe una respuesta correcta). Indique cuál(es) de las siguientes afirmaciones es/son verdadera(s):

- a) Cada expresión debe tener un tipo, que se calcula antes de evaluar la expresión mediante un proceso llamado inferencia de tipos.
- b) La inferencia de tipos precede siempre a la evaluación.
- c) Los errores de tipos no ocurren durante la evaluación, pero sí pueden darse en la inferencia de tipos.

d) Todas son verdaderas.

Ejercicio 2.10. En Haskell, las listas deben ser homogéneas, mientras que las tuplas pueden ser heterogéneas. VERDADERO o FALSO. Justifique su respuesta.

Ejercicio 2.11. La aridad de una tupla es el número de componentes. Las tuplas pueden tener aridad 0, aridad 2 y aridad n , pero no pueden tener aridad 1. VERDADERO o FALSO. Justifique su respuesta.

Ejercicio 2.12. Una tupla con aridad 2 tiene un tipo diferente a una tupla con aridad 3 (aunque contengan el mismo tipo de valores). VERDADERO o FALSO. Justifique su respuesta.

Ejercicio 2.13. Las listas pueden ser infinitas, mientras que las tuplas deben tener una aridad finita para garantizar que los tipos de tuplas siempre puedan calcularse antes de la evaluación. VERDADERO o FALSO. Justifique su respuesta.

Ejercicio 2.14. Un tipo polimórfico es un tipo que contiene una o más variables de tipo, y una función que tiene un tipo polimórfico se llama función polimórfica. VERDADERO o FALSO. Justifique su respuesta.

Ejercicio 2.15. En Haskell, las variables de tipo se escriben usualmente en minúscula (por ejemplo, `a`, `b`). Explique qué significa que una función tenga una variable de tipo en su firma (por ejemplo, `id :: a -> a`) y por qué eso indica que la función es polimórfica.

Ejercicio 2.16. Explique qué es la parcialización y la currificación, y cómo se relacionan entre sí.

Ejercicio 2.17. Defina el proceso de evaluación perezosa en un programa funcional. ¿Qué ventajas ofrece sobre una evaluación ansiosa?

Ejercicio 2.18. ¿Qué significa que una función sea de orden superior? Justifique su respuesta.

Ejercicio 2.19. Las mónadas nos permiten resolver diferentes problemas. ¿Cuáles serían? Justifique cómo estos problemas afectan a un lenguaje funcional.

Ejercicio 2.20. Defina y describa, mediante ejemplos, los constructores de tipo versus los constructores de datos utilizados en Haskell. Incluya ejemplos con `data` usando `1stlisting`.

2.2. Ejercicios básicos

Ejercicio 2.21. Calcular la media aritmética de tres valores numéricos.

Ejercicio 2.22. Calcular el máximo entre 3 números (puede utilizar la función predefinida `max`).

Ejercicio 2.23. Definir una función tal que reciba un par (x, y) y retorne el cuadrante. El par no está sobre los ejes x ni y .

Ejercicio 2.24. Definir la función `igualesTres` que verifica que tres elementos x , y , z son iguales.

Ejercicio 2.25. Definir la función `diferentesTres` que verifica que tres elementos x , y , z son diferentes.

Ejercicio 2.26. Definir la función `esPar` que verifica si un número es par.

Ejercicio 2.27. Definir la función `esPrimo` que verifica si un número es primo.

Ejercicio 2.28. Definir la función `esBisiesto` que verifica si un año es bisiesto.

2.3. Uso de guardas, patrones y tipos definidos por el usuario

Ejercicio 2.29. Definir las siguientes funciones utilizando guardas, patrones y datos definidos por el usuario. En ningún caso, utilizar funciones predefinidas de Haskell.

A. Calcular la negación, la conjunción y la disyunción usando patrones con valores.

B. Determinar si un número es par.

C. Con base en el puntaje obtenido en un examen (0–100), se desea saber la condición del alumno (aprobado o desaprobado).

- D. Mezcla de colores: dado el porcentaje de dos colores primarios, obtener un color secundario (y sucesivos).
- E. Clasificar personas por edad:
- Menor: < 13
 - Adolescente: 13 a 17
 - Adulto: 18 a 69
 - Anciano: > 69
- F. Clasificar personas (18 o más) por nivel de estudios: primarios, secundarios o superiores.
- G. Clasificar triángulos por lados: equilátero, isósceles, escaleno o “no es triángulo” (desigualdad triangular).
- H. Definir el tipo `Figura` (círculo, rectángulo, cuadrado) y `area :: Figura -> Float`.